

SENAC-RS

CURSO DE ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

FRAMEWORKS WEB

PEDIDO DE DIÁRIAS EM ÓRGÃOS PÚBLICOS

*Desenvolvimento de uma Aplicação Fullstack para Automação de Processos
Administrativos*

Autor: Adriano Nunes dos Santos

Professor: Luis Henrique Ries

Canoas, Rio Grande do Sul

Julho de 2026

1. Introdução

Este relatório descreve o desenvolvimento de uma aplicação fullstack voltada para a gestão e o cálculo de diárias no setor público. Utilizando o framework NestJS no back-end e React com Vite no front-end, o projeto aplica princípios de arquitetura em camadas e desacoplamento de sistemas para garantir que a lógica de negócio seja independente, segura e escalável. A escolha do tema justifica-se pela complexidade intrínseca das regras de indenização por deslocamento de servidores, que dependem diretamente de legislação variável.

2. Definição do Problema

O processo de solicitação de diárias em prefeituras, câmaras, tribunais de justiça, ministérios públicos e órgãos correlatos é normalmente estigmatizado por falta de padronização. Atualmente, muitos desses processos são realizados manualmente ou por ferramentas rudimentares (como planilhas eletrônicas) sem validação automática ou qualquer tipo de auditoria, o que acarreta em:

- **Inconsistência nos valores:** erros manuais ao aplicar acréscimos/decrécimos conforme a localidade, cargo ou horário.
- **Lentidão no fluxo:** a necessidade de conferência humana minuciosa para cada cálculo atrasa o ressarcimento do servidor.
- **Dificuldade de auditoria:** sistemas sem regras de negócio rígidas dificultam o controle fiscal de gastos públicos.

3. Solução Proposta

A solução desenvolvida é um ecossistema computacional composto por uma API (Application Programming Interface) que centraliza a inteligência do cálculo de diárias e uma SPA (Single Page Application) que serve como interface gráfica interativa. O sistema permite que a administração cadastre os pedidos e receba, em tempo real, o valor exato a ser pago, já processado conforme as regras vigentes.

3.1 Requisitos Funcionais

- **RF01 - Cálculo Automático:** O sistema deve calcular o valor total com base no cargo do servidor.
- **RF02 - Adicional de Destino:** Deve ser aplicado um acréscimo de 30% para destinos especiais como Porto Alegre e Brasília.
- **RF03 - Fator Pernoite:** O sistema deve reduzir o valor em 50% caso a viagem não possua pernoite (bate-volta).
- **RF04 - Validação de Dados:** O sistema deve impedir o processamento de cargos que não estejam previstos na legislação cadastrada.

- **RF05 - Cadastro de Servidores:** O sistema deve permitir o gerenciamento completo (CRUD) dos servidores públicos, contendo matrícula única, nome e cargo.
- **RF06 - Relacionamento de Dados:** Cada diária gerada deve ser obrigatoriamente vinculada a um servidor existente no banco de dados.
- **RF07 - Paginação de Resultados:** A listagem de diárias deve ser paginada no banco para otimizar o tempo de resposta e o tráfego de dados.
- **RF08 - Formulário de Cadastro Interativo (Front-end):** A interface deve disponibilizar campos para preenchimento de destino, cargo, ID do servidor e opção de pernoite, realizando o envio assíncrono dos dados.
- **RF09 - Tratamento de Exceções Visual:** O front-end deve capturar erros de validação disparados pelo back-end e exibi-los em um componente visual de alerta amigável ao usuário.

4. Arquitetura do Sistema

A aplicação foi estruturada seguindo o padrão de Arquitetura em Camadas (Layered Architecture) no back-end e arquitetura baseada em Componentes Reutilizáveis no front-end, o que facilita a manutenção e a testabilidade do código de forma totalmente desacoplada.

4.1 Divisão de Camadas (Back-end)

- **Camada de Controladores (Controllers):** Responsável por expor os endpoints, receber as requisições HTTP externas e validar o formato dos dados de entrada.
- **Camada de Serviços (Services/Domain):** Onde reside o *core* da aplicação. Toda a lógica de cálculo de porcentagem, faturamento e validação de cargos foi isolada nesta camada, mantendo-a pura e independente.
- **Camada de Entidades (Entities / Prisma Schema):** Define os modelos de dados (Servidor e Diária) que são persistidos e transitam pela aplicação.

4.2 Decisões Técnicas

A escolha pelo **NestJS** foi motivada pela sua arquitetura "opinativa", que impõe uma organização modular natural através de Injeção de Dependências. O uso de **TypeScript** trouxe segurança por meio de tipagem estática, reduzindo erros em tempo de execução, especialmente críticos em sistemas financeiros e administrativos públicos.

A persistência real de dados utiliza o **Prisma ORM** como ferramenta de mapeamento objeto-relacional e o banco de dados **SQLite**. O SQLite foi escolhido por sua portabilidade e eficiência em ambientes de desenvolvimento, enquanto o Prisma garantiu a segurança das consultas por meio de tipagem estática gerada automaticamente a partir do schema do banco, eliminando o risco de *SQL Injection*.

Para o desenvolvimento do Front-end do Sistema de Gestão de Diárias em Órgãos Públicos (SGDOP), foi implementado o modelo de renderização **CSR/SPA** através da biblioteca **React** e

a ferramenta de build **Vite**. O isolamento de responsabilidades é completo: o front-end gerencia o estado da interface e o roteamento interno no navegador do usuário, consumindo os endpoints da API de forma assíncrona via requisições HTTP utilizando a biblioteca **Axios**, sob políticas estritas de CORS.

5. Modelagem de Domínio e Regras de Negócio

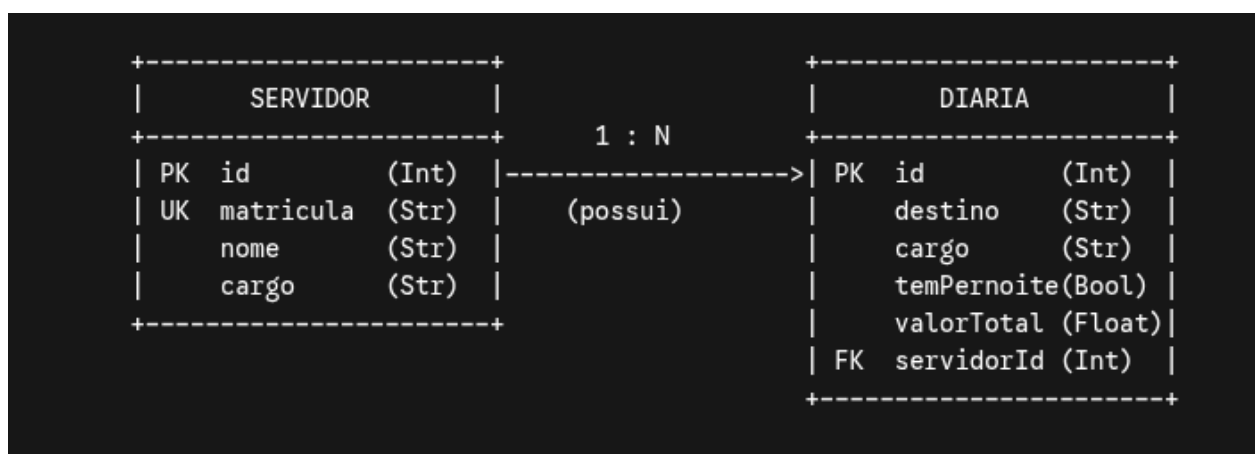
Para este projeto, as regras de negócio foram parametrizadas da seguinte forma:

- **Tabela de Valores por Cargo:**
 - Operacional: R\$ 200,00
 - Técnico: R\$ 350,00
 - Gestão: R\$ 600,00
- **Lógica de Acréscimo:** Utilização de uma lista de destinos especiais para a aplicação de um multiplicador de 1.3 (+30%).
- **Lógica de Redução:** Aplicação de fator multiplicador de 0.5 (-50%) para pedidos informados com *temPernoite* = *false*.

6. Persistência e Modelagem Relacional

Para suportar as regras de negócio, o banco de dados foi modelado de forma relacional com duas tabelas principais e um relacionamento de 1 para Muitos (1:N): um servidor pode possuir múltiplas diárias, mas uma diária pertence obrigatoriamente a apenas um servidor.

- **Tabela Servidor:** *id* (auto-incremento), *matricula* (texto único), *nome* (texto) e *cargo* (texto).
- **Tabela Diaria:** *id* (auto-incremento), *destino* (texto), *cargo* (texto), *temPernoite* (booleano), *valorTotal* (decimal) e *servidorId* (chave estrangeira integrada).



6.1 Indexação de Performance

Visando a otimização de busca nas tabelas, foi adicionado um índice explícito de banco de dados (`@@index([servidorId])`) na tabela *Diaria* através do Prisma. Esse índice armazena de forma ordenada os ponteiros de relacionamento de chaves estrangeiras, impedindo que o banco de dados realize varreduras completas na tabela (*Full Table Scans*) ao cruzar os dados, mantendo o tempo de pesquisa otimizado mesmo com alta volumetria de dados.

7. Mecanismos Fortes de Segurança

A aplicação foi blindada seguindo boas práticas de segurança digital através de cinco camadas de proteção:

- **Rate Limiting (Proteção contra DoS/Força Bruta):** Implementação do `@nestjs/throttler` restringindo requisições em massa (limite estrito de 10 requisições a cada 60 segundos por IP), evitando a sobrecarga e indisponibilidade do servidor.
- **Cabeçalhos de Segurança (Helmet):** Integração do middleware `helmet` no ponto de entrada do back-end para a injeção automatizada de 15 cabeçalhos HTTP protetores contra vulnerabilidades como XSS (*Cross-Site Scripting*), *Clickjacking* e *MIME-sniffing*.
- **Cross-Origin Resource Sharing (CORS):** Ativação global e restrita do mecanismo de proteção CORS no NestJS para controlar e mitigar quais origens e domínios de clientes externos estão autorizados a consumir os dados da API.
- **Sanitização e Antipoluição de Dados (Whitelist Pipes):** Configuração do `ValidationPipe` nativo com a propriedade `whitelist: true`. Isso força a API a descartar e ignorar qualquer propriedade maliciosa ou dado intruso enviado no formato JSON que não esteja explicitamente previsto nos DTOs (*Data Transfer Objects*).
- **Isolamento de Credenciais de Infraestrutura:** Proteção da string de conexão do banco de dados e portas operacionais por meio de variáveis de ambiente no arquivo `.env`, blindado localmente e mapeado no `.gitignore` para impedir o vazamento inadvertido de dados sensíveis em repositórios públicos como o GitHub.

8. Estratégias de Performance e Escalabilidade Fullstack

O sistema foi planejado para trabalhar com o menor consumo de recursos possível, entregando tempos de resposta rápidos e uma renderização fluida em todas as pontas:

- **Paginação de Consultas no Banco de Dados:** O endpoint de listagem de diárias aceita os parâmetros `page` e `limit`. Utilizando as cláusulas `skip` e `take` do Prisma ORM, o SQLite processa e entrega apenas pequenos blocos de registros por vez na

memória, reduzindo o consumo de hardware e o tamanho do arquivo trafegado em rede.

- **Cache de Dados em Memória RAM (Back-end):** Integração do *@nestjs/cache-manager* na rota de listagem. Os dados consultados do banco permanecem em cache na memória volátil do servidor. Requisições repetidas subsequentes ignoram o banco e são respondidas diretamente em 0 milissegundos, poupando processamento de CPU.
- **Gerenciamento de Estado em Memória RAM (Front-end):** A interface utiliza estados reativos locais (*useState*). A aplicação retém os dados da tabela em memória no navegador e apenas dispara novas requisições de listagem caso ocorra uma mutação confirmada no sistema (como o sucesso de um novo cadastro), reduzindo chamadas síncronas redundantes.

9. Tratamento de Exceções Global Unificado

A aplicação implementa uma arquitetura de tratamento de erros em ambas as pontas. Caso uma requisição envie dados inválidos (como um destino em branco, ou um ID de servidor inexistente no banco), o mecanismo *ValidationPipe* do NestJS captura a falha e dispara uma resposta estruturada (*BadRequestException*).

No front-end (React), as chamadas do *Axios* são encapsuladas em estruturas de controle *try/catch*. Ao interceptar uma falha de validação ou de rede vinda do back-end, o front-end impede a quebra silenciosa da aplicação, extrai as mensagens de erro geradas na API e as renderiza dinamicamente em um componente de alerta vermelho amigável na tela do usuário, garantindo resiliência de software.

10. Ferramentas Utilizadas

- **Framework NestJS:** Para a estrutura robusta, modular e escalável do backend.
- **Biblioteca React & Vite:** Para a construção da interface SPA veloz, componentizada e reativa.
- **Prisma ORM:** Ferramenta de mapeamento objeto-relacional (ORM) para comunicação tipada e segura com o banco.
- **Banco de Dados SQLite:** Banco de dados relacional local leve, ágil e portátil.
- **Pacotes de Segurança (Helmet & Throttler):** Módulos integrados para injeção de cabeçalhos protetores e Rate Limiting.
- **Cache Manager:** Módulo utilizado para a implementação de cache em memória RAM.
- **Swagger (OpenAPI):** Implementado para a documentação interativa e testes dos endpoints da API.
- **Prisma Studio:** Interface visual de administração e auditoria de dados do banco.
- **GitHub:** Utilizado para controle de versão, histórico de alterações e repositório de código.

11. Uso Crítico de Ferramentas de IA

A Inteligência Artificial foi utilizada de forma estratégica durante o desenvolvimento. O uso ficou restrito a sanar dúvidas técnicas pontuais e nos seguintes tópicos:

- **Estruturação Arquitetural:** Auxílio na definição de módulos e injeção de dependência do NestJS.
- **Aceleração de Código:** Geração de boilerplate (código base estrutural) para as entidades, controladores e estilização inline do front-end.
- **Análise Crítica:** A decisão final sobre as regras de negócio e a supervisão da lógica de cálculo foram mantidas sob controle humano, garantindo que o código gerado atendesse especificamente ao domínio público solicitado (com base em regras reais observadas em sistemas ERP voltados a órgãos públicos brasileiros, como os desenvolvidos pela empresa Thema Informática para clientes como o Tribunal de Justiça do Estado de Santa Catarina).

12. Trabalhos Futuros e Escalabilidade (Melhorias Planejadas)

Como perspectivas e próximos passos para a evolução da plataforma em um cenário de produção real, identificam-se as seguintes demandas de implementação:

- **Módulo de Autenticação e Autorização Seguro (JWT):** Implementação de uma tela de login no front-end integrada a um sistema de autenticação por tokens JWT (*JSON Web Tokens*) protegidos no back-end. As senhas dos usuários seriam criptografadas no banco utilizando algoritmos de hashing forte (*bcrypt*), exigindo o envio do token no cabeçalho das requisições para a liberação dos serviços.
- **Controle de Acesso Baseado em Perfis (RBAC - Role-Based Access Control):** Integração de guardas de segurança (*Guards*) nas rotas da API para segmentar privilégios. Sob essa regra, servidores comuns teriam permissão restrita apenas para solicitar diárias, enquanto perfis de nível "Gestor" possuiriam acesso exclusivo para aprovar ordens de pagamento, auditar históricos e gerenciar o cadastro de funcionários.
- **Barra de Busca e Filtros Avançados:** Implementação de campos de pesquisa na interface gráfica para permitir a filtragem de diárias por nome do servidor, intervalo de datas ou destinos específicos, executados diretamente via parâmetros na consulta ao banco.

13. Considerações Finais

O desenvolvimento do projeto Pedido de Diárias em Órgãos Públicos permitiu consolidar com excelência os conhecimentos de Frameworks Web e arquitetura de software de alto nível. A evolução do protótipo isolado para uma aplicação fullstack completa e desacoplada demonstra

que é possível modernizar processos burocráticos e complexos do setor público através de soluções tecnológicas padronizadas, visuais e seguras.

A principal contribuição desta versão final do trabalho é a entrega de uma aplicação resiliente e funcional de ponta a ponta. O front-end reativo gerencia o fluxo de cadastro e o exibe de forma clara ao usuário, enquanto o back-end processa regras complexas sob uma infraestrutura blindada com cinco camadas de segurança cibernética e otimizada por estratégias de indexação de banco, paginação e cache em memória (0ms). O projeto estabelece uma fundação sólida e estruturada, pronta para receber novos módulos de controle de acessos em futuras expansões.